Progress Documentation - Activity 3

Eventer: A High School Event Management System (HSEMS)
Group 2 | BSIT 3-2

Polytechnic University of the Philippines - San Pedro Campus

I. Team & Project Links

A. Team

Member	Role/Responsibility
Manzon, Mark Anthony D.	Project Manager
Magluyan, Allen Adrian A.	Frontend Developer
Babas, Joseph Emmanuel V.	Backend Developer & Database Administrator
Lumayno, James David L.	Quality Assurance
Manaloto, Neil Emmiel F.	Technical Documentation

Note: Team members may take on multiple roles depending on project priorities and requirements.

B. Project Links

- **Repository Link**: https://github.com/JosephEmmanuelBabas/School_Event_Management_Web_Appli.git

- **Tracker Link**: https://docs.google.com/spreadsheets/d/1BClCa9rEs9o-D9Rp3tUZVhECipJ234zX/edit?usp=sharing&ouid=101490909816216416819&rtpof=true&sd=true

II. System Overview

A. Purpose

The High School Event Management System is a web-based application that centralizes school event information that is usually scattered across paper forms, spreadsheets, and messaging apps. It covers events, schedules, venues, staff assignments, participant attendance, and audit logs in one system.

(This system implements the approved Activity 2 proposal, including its requirements, business rules, ERD, and relational schema.)

B. Scope

1. **Event Management** - name, description, date, start/end time, status, organizer, venue, and assigned staff.
2. **Attendance Management** - event-based only
3. **Scheduling Management** - manage event schedules, view scheduled activities, and view schedule conflicts.
4. **Venue and Staff Management** - maintain records of available venues and assigned staff.
5. **User and Account Management** - role-based access control implemented for users.
6. **Analytics** - provides basic analytics related to events and attendance.
7. **Administrative Dashboard** - a dashboard that provides an overview of relevant event, attendance, user, and system information.
8. **Audit Logs** - maintain records of significant user activities.
9. **Web-Based Platform** - accessible through web browsers.

C. Functional Requirements

#	Feature	Description	Priority
1	User Authentication (RBAC)	Allows authorized users to securely log in and provides access based on their assigned role.	High
2	Account Management	Allows users to manage basic account information such as name, username,	Medium

		password, and profile details.	
3	Attendance Management	Allows authorized users to record, view, update, and monitor attendance for school events.	High
4	Scheduling Management	Allows organizers to create and manage event dates, times, schedules, and identify possible schedule conflicts.	High
5	Venue / Staff Management	Allows organizers to manage available venues and assign staff members to specific events.	Medium
6	Analytics (Attendance, Event)	Provides basic summaries and statistics regarding event participation and attendance.	Medium
7	Admin Dashboard	Provides the Super Admin with an overview of events, attendance, users, and other relevant system information.	High
8	Audit Logs	Records important actions performed within the system for monitoring and accountability.	High
9	User Management	Allows the Super Admin to create, update, deactivate,	Medium

		and manage authorized system users and their roles.	
10	Event Lists	Provides a centralized list of upcoming, ongoing, completed, and other school events with relevant event information.	Low
11	Participant/Student Records Management	Allows authorized users to add, view, update, and manage participant (student) records used for attendance tracking.	Medium

D. Tech Stack

Layer	**Technology / Framework**
Frontend	React 19, Node.js v24.22.0
Backend	Java 25, Spring Boot 4.1 (Web MVC, Data JPA / Hibernate, Security, Validation), Maven wrapper
Database	PostgreSQL 18.6, Flyway 12 migrations

III. User Roles

A. Application Roles and Database Role Mapping

Application Role	**Responsibilities in the Application**	**Functions and Data Access**	**Intended Database Role**
Super Admin	Manages the overall system, user accounts, and roles; monitors system activity.	Create, read, update, and deactivate on user_account; read on role, event, attendance, participant, venue,	event_app

		and audit_log; read and update own account; dashboard and reports. Restricted to these functions by the application (Spring Security), not by a separate database role.	
Event Organizer	Creates and manages events, schedules, venue and staff assignments, and attendance.	Create, read, update, and delete on event, event_schedule, event_assignment, participant, attendance, and venue; read and update own account (profile and password). All organizers can manage all events (Business Rule 11); organizer_id records ownership only.	event_app
Application Runtime (system account)	The account the application uses to connect to PostgreSQL.	Read on role and user_account (login and RBAC checks), the operations above on operational tables, and read/insert on audit_log.	event_app
Schema Owner (migrations)	Creates and changes tables when the backend starts (Flyway).	Owns schema school_event_management_db; runs migrations V1 to V3. Not used for normal application operations.	event_owner

B. Database Accounts

Database Role	Type	Login Account	Intended Privileges
event_owner	Login role (schema owner, not a superuser)	event_owner	Owns schema school_event_management_db; CREATE/ALTER on its objects through Flyway migrations only.
event_app	Login role (application runtime, not a superuser)	event_app	USAGE on schema school_event_management_db; SELECT on role; SELECT, INSERT, UPDATE on user_account (no DELETE, accounts are only deactivated); SELECT, INSERT, UPDATE, DELETE on venue, participant, event, event_schedule, event_assignment, attendance; SELECT, INSERT on audit_log (append-only); USAGE on sequences. No CREATE, no CREATEDB, no CREATEROLE.

IV. Development Progress Log

Date	Milestone / Task	Member(s)	Key Commits
[date]	description	All or [name/s]	- (Empty if not code related)

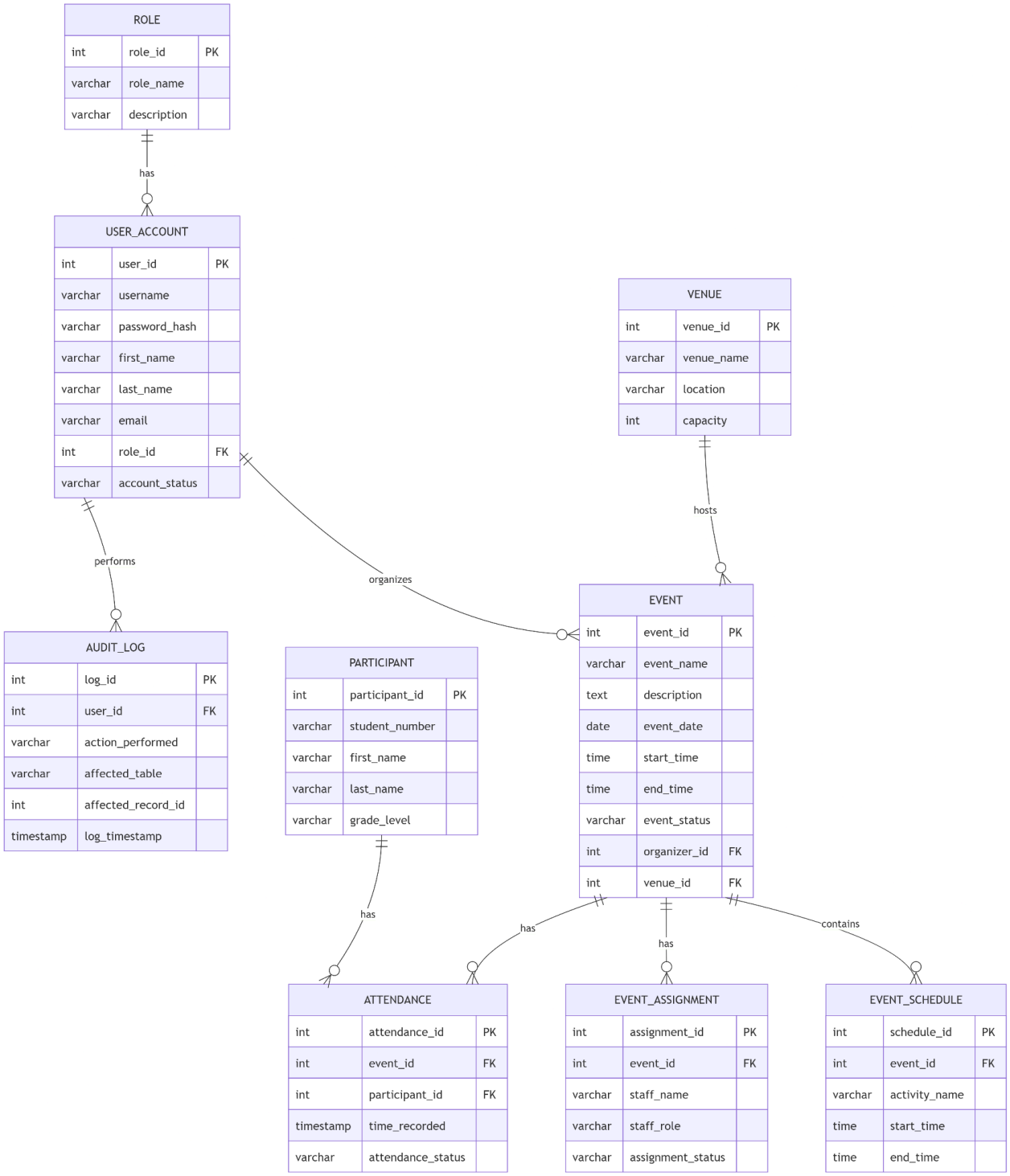
[date]	Created group repository and added all members as collaborators	[name]	[hash/message] (you copy the hash and commit message from our repo or you source control in VS Code)

V. Database Implementation

A. Database Summary

Item	**Details**
Database Platform	PostgreSQL 18.6
Database Name	grp02_event_management_db
Schema Name	school_event_management_db
Number of Tables	9
Database Owner	Babas, Joseph Emmanuel V.

B. Entity Relationship Diagram



C. Schema Summary

Table	Primary Key	Foreign Keys	Key Constraints
role	role_id	-	role_name UNIQUE
user_account	user_id	role_id → role	username UNIQUE; email UNIQUE (nullable); account_status DEFAULT 'Active'
venue	venue_id	-	venue_name UNIQUE
event	event_id	organizer_id → user_account; venue_id → venue (both NOT NULL)	event_status CHECK (Upcoming, Ongoing, Completed, Cancelled)
event_schedule	schedule_id	event_id → event	-
event_assignment	assignment_id	event_id → event	assignment_status (Confirmed, Pending, Cancelled)
participant	participant_id	-	student_number UNIQUE
attendance	attendance_id	event_id → event; participant_id → participant	UNIQUE (event_id, participant_id); attendance_status CHECK (Present, Late, Absent); time_recorded DEFAULT CURRENT_TIMESTAMP
audit_log	log_id	user_id → user_account	action_performed, affected_table, affected_record_id, log_timestamp NOT NULL

D. Migrations / SQL Scripts and Seed Data

File	Purpose
[path/01_schema.sql]	Creates the schema and tables
[path/02_constraints.sql]	Adds constraints and indexes
[path/03_roles.sql]	Creates group roles and login accounts (placeholder passwords)
[path/04_seed.sql]	Loads sample data: [number] roles, [number] users, [number] venues, [number] events, [number] participants, [number] attendance records

E. Changes from the Approved Design

Change	Reason
Add if there's changes	

VI. Application Evidence

A. Screenshots

[Insert application screenshots with captions (e.g. Figure #: caption)]

B. CRUD Test Results

Entity	Create	Read	Update	Delete	Result
User account	Admin: Pass Organizer: 403	Admin: Pass	Admin: Pass	Deactivated instead of deleted	Pass
Venue	Organizer: Pass Admin: 403	Both: Pass	Organizer: Pass	Organizer: Pass	Pass
Event	Organizer: Pass	Both: Pass	Organizer: Pass	Organizer: Pass	Pass

Event schedule	Organizer: Pass	Both: Pass	Organizer: Pass	Organizer: Pass	Pass
Event assignment	Organizer: Pass	Both: Pass	Organizer: Pass	Organizer: Pass	Pass
Participant	Organizer: Pass	Both: Pass	Organizer: Pass	Organizer: Pass	Pass
Attendance	Organizer: Pass	Both: Pass	Organizer: Pass	Organizer: Pass	Pass

Role and audit_log are not included in CRUD testing because role is seeded data and audit_log is generated automatically by the system.

VII. Setup and Configuration

A. Installation Steps

1. Install the prerequisites: PostgreSQL 18.x, JDK 25, Maven (not needed; the project includes the Maven wrapper `mvnw`), Node.js 20.19+ / 22.12+ / 24 (LTS), and Git.

2. Clone the repository, then go into the project folder:
    - git clone https://github.com/JosephEmmanuelBabas/School_Event_Management_Web_Appli.git
    - cd School_Event_Management_Web_Appli

3. Install dependencies:
    - Backend: ./mvnw clean install -DskipTests (Windows PowerShell: .\mvnw.cmd clean install -DskipTests). This is optional, because ./mvnw spring-boot:run downloads them on first start.
    - Frontend: cd frontend, then npm install (use npm.cmd in PowerShell if scripts are disabled)

4. Configure the environment:
    - Backend: copy school_event_management/src/main/resources/application-local.properties.example to school_event_management/src/main/resources/application-local.properties and enter your local values.
    - Frontend: no .env file is needed. The Vite dev server proxies /api to http://localhost:8080.

5. Create the database, schema, extension, and database users by running the setup script as the postgres superuser. It creates grp02_event_management_db; there is no separate CREATE DATABASE step:
    - psql -U postgres -h localhost -f docs/database-setup.sql
    - Choose your own passwords for event_owner and event_app when prompted.

6. Run the schema migrations: they run automatically (Flyway V1 to V3: create tables, seed roles, app privileges) the first time the backend starts in step 8. No manual command is needed.

7. Database role setup: already done by docs/database-setup.sql in step 5 (event_owner for migrations, event_app for the application). Put your own local passwords in application-local.properties (spring.flyway.password for event_owner, spring.datasource.password for event_app).

8. Start the backend (Terminal 1):
    - cd school_event_management
    - ./mvnw spring-boot:run (Windows PowerShell: .\mvnw.cmd spring-boot:run)
    - Wait for "Successfully applied 3 migrations" and "Started SchoolEventManagementApplication".

9. Load the initial data: there is no sample-data script. The roles are seeded by migration V2, and the first Super Admin and Event Organizer are created automatically on the first start from the app.bootstrap.* values in application-local.properties.

10. Start the frontend (Terminal 2), then open the app:
    - cd frontend
    - npm run dev
    - Open http://localhost:5173 and log in with the admin from application-local.properties.

B. Environment Variables

Note: Never expose secret keys and password

(This project has no .env files. Settings are in application.properties (committed, no secrets) and application-local.properties (Git-ignored, holds passwords).)

Backend (application.properties):

```
spring.datasource.url=jdbc:postgresql://localhost:5432/grp02_event_management_db?cur
rentSchema=school_event_management_db
spring.datasource.username=event_app
spring.flyway.user=event_owner
```

spring.flyway.schemas=school_event_management_db

Backend (application-local.properties, local only, not committed):
spring.datasource.password=<event_app password>
spring.flyway.password=<event_owner password>
app.bootstrap.admin.username=admin
app.bootstrap.admin.password=<at least 10 characters>
app.bootstrap.organizer.username=organizer1
app.bootstrap.organizer.password=<at least 10 characters>

Mapping to the template's variables:
DB_HOST=localhost, DB_PORT=5432, DB_NAME=grp02_event_management_db, DB_SCHEMA=school_event_management_db (all inside the datasource URL), DB_USER=event_app, DB_PASSWORD=spring.datasource.password.

Frontend:
None. The API URL is not configured in a .env file; vite.config.js proxies /api to http://localhost:8080 (frontend runs on port 5173).

C. Database User and Role Setup

Run docs/database-setup.sql (Step 5) as the postgres superuser. It creates the database, the schema school_event_management_db owned by event_owner, the btree_gist extension, and two login roles:

- event_owner: owns the schema; used by Flyway to create and change tables.
- event_app: the application runtime login; read/write on data tables, read-only on role, no DELETE on user_account, and only SELECT and INSERT on audit_log. It cannot change the schema.

The privileges for event_app are applied by migration V3__app_privileges.sql when the backend first starts.

Set your own local passwords when prompted and put them in application-local.properties. The application connects as event_app (the runtime login) and never as a superuser.
Sample application accounts: the Super Admin and Event Organizer are created on first start from the app.bootstrap.* values you set locally (example usernames: admin, organizer1). Their passwords are not stored in the repository.

VIII. Issues and Resolutions

#	Issue	Cause	Resolution
1	[e.g., Foreign key error when loading seed data]	[Wrong insert order]	[Reordered the seed inserts]
2	[e.g., Connection refused]	[Service not running / wrong port]	[Started the PostgreSQL service and confirmed port 5432]
3	[issue]	[cause]	[solution]

IX. Cross-Group Verifications

A. Projects Tested by Our Group

Project Tested	Repository / Branch	Setup Result	Roles Checked	Issues Found	Feedback Sent
Group [#] – [name]	[URL / branch]	[Success / Success with issues / Failed]	[roles and result]	[summary]	[date]
Group 9 - Batumbakal Gym Membership Management System	https://github.com/louisandreidg/Batumbakal-Gym	Success	gym_app - Success gym_readonly - Success	No Issue	11/05/2026
Group 3 - PAWPAW: Veterinary Appointment and Pet Management System	https://github.com/rubiakylashane/DBA_PAWPAW.git	Success	Pet Owner - Success Staff - Success Veterinarian - Success Admin - Success	No Issue	11/05/2026

Group 7 - Applicant Management System (AMS)	https://github.com/mang2mas/grp07-applicant-management-system	Failed			
Group 11 - ReserveIT-Classroom-and-Material-Reservation-System	https://github.com/sev-villanueva/ReserveIT-Classroom-and-Material-Reservation-System-	Success	Student / Facility - Success Staff - Success Admin - Success	No Issue	11/05/2026
Group 1 - DropBoxed: An Environmental Complaint Application for Household Water Quality Monitoring in San Pedro	https://github.com/joshxaaaaaa/GRP1_Dropboxed.git	Success	Admin - Success Resident - Success Visitor - Success	No Issue	11/06/2026
Group 6 - Car Rental Management System	https://github.com/123airabels/Car_Rental_Management_System	Success	System Administrator - Success Administrator - Success Rental Staff -Success Finance - Success Customer - Success	No Issue	11/06/2026
Group 4 - HealthBridge Clinic Management System	https://github.com/kishaop/GRP03_PROJECTDB	Success	Patient - Success Staff - Success	No Issue	11/06/2026

			Doctor - Success Admin - Success		

B. Feedback Received on Our Project

Testing Group	Result	Issue Reported	Fix Made
Group [#]	[Pass / Pass with issues / Fail]	[issue]	[fix]
Group 3 - PAWPAW: Veterinary Appointment and Pet Management System	Pass	N/A	N/A
Group 1 - DropBoxed: An Environmental Complaint Application for Household Water Quality Monitoring in San Pedro	Pass	N/A	N/A

X. AI Usage Declaration

AI Tool	Task Performed	How the Group Validated or Corrected the Output
Gemini	[e.g., Generated SQL scripts from the approved schema]	[Compared against the ERD and data dictionary; tested on a clean database]
Gemini/Claude	[e.g., Generated application code]	[Reviewed and tested role restrictions; removed hard-coded credentials]

Gemini/Claude	[e.g., Troubleshooting and documentation drafts]	[Verified fixes locally; tested the README steps on another machine]